- What is Amazon Bedrock AgentCore ? Amazon Bedrock AgentCore is an **agentic platform** to build, deploy, and operate highly capable agents securely at scale.

Amazon Bedrock AgentCore service provides a set of capabilities that can help you transition from POC built on agentic AI to production ready systems.

- Amazon Bedrock AgentCore and Amazon Bedrock Agents are two different services in the AWS GenAI Ecosystem.
    - Amazon Bedrock Agents : is a fully managed service that allows you to build and configure autonomous agents without managing infrastructure or writing custom code.
    - Amazon Bedrock AgentCore : is a comprehensive set of services designed to deploy and operate AI agents securely at scale using any AI framework and Large Language Model.
- Core Problems AgentCore Fixes:
    1. LLMs are stateless LLMs don't remember or plan. 👉 AgentCore adds: Memory Context tracking Multi-step reasoning
    2. Tool usage is hard LLMs don't naturally call APIs reliably. 👉 AgentCore: Defines tools (APIs, Lambda, DBs) Lets the model choose and invoke them safely
    3. No built-in orchestration Multi-step workflows = messy code 👉 AgentCore: Handles step-by-step execution Automatically chains actions
    4. Unpredictability of LLMs LLMs hallucinate, loop, or fail silently 👉 AgentCore: Adds guardrails Retry logic Structured execution
    5. Scaling & Productionization A demo ≠ production system 👉 AgentCore provides: Observability Logging Integration with AWS ecosystem

    🧩 Simple Mental Model

    ```
    User Request
        ↓
    AgentCore (brain + workflow engine)
        ↓
    Foundation Model (Claude / Titan)
        ↓
    Tools (APIs, DBs, Lambda)
        ↓
    Final Action / Response
    ```

🧠 Real-World Example Without AgentCore:

```
You manually code:
    "If user asks X → call API A"
    "Then parse response → call API B"
```

```
    “Store state somewhere”
    “Retry if failure”

With AgentCore:
    You define:
        Goal: “Plan a trip”
        Tools: Flight API, Hotel API

    👉 AgentCore handles the rest.
```

⚖️ Honest Take (Important) AgentCore is powerful—but not magic:

```
👍 Strengths
    Reduces engineering complexity
    Standardizes agent design
    Integrates deeply with AWS

👎 Limitations
    Less flexible than custom frameworks (e.g., LangChain-style setups)
    Still dependent on LLM reliability
    Can feel like a “black box” in complex workflows
```

- Once we built AgenticAI (RAG or GenerativeAI) applications, deploying them to production wrt scalability, observability monitoring is a big challenge, thats where Bedrock AgentCore come into.

Core Services in Amazon Bedrock AgentCore (GO PRIM)

(Refer slides : 11-bedrock-agentCore-capabilities.png, 12-bedrock-agentCore-capabilities.png, 13-bedrock-agentCore-capabilities.png, 14-bedrock-agentCore-capabilities.png)

AgentCore is a modular, composable platform. You can use these services together or independently. They work with any agent framework (LangGraph, CrewAI, LlamaIndex, Strands, etc.) and any foundation model.

1. Gateway
2. Observability
3. Policy
4. Runtime
5. Identity
6. Memory

Additional Built-in Capabilities (Often Treated as Core Tools) 7. Code Interpreter — Secure tool for executing Python code, data analysis, chart generation, etc. 8. Browser Tool — Secure browser for web navigation, data extraction, and interacting with websites.

#	Component	Description	Key Purpose
1	AgentCore Runtime	Executes agent workflows and manages lifecycle	Foundation for running agents in production

#	Component	Description	Key Purpose
2	AgentCore Memory	Stores context, history, and embeddings	Makes agents context-aware and personalized
3	AgentCore Gateway	Handles tool/API access and routing	Secure tool integration and discovery
4	AgentCore Identity	Manages authentication and authorization	Enterprise-grade security and authorization
5	AgentCore Observability	Provides logging, tracing, and metrics	Monitoring, debugging, and performance insights
6	AgentCore Policy	Enforces policies and safety controls	Safety, compliance, and controlled agent behavior

1. **AgentCore Runtime**:
   Amazon Bedrock AgentCore Runtime is the foundational and central component of Amazon Bedrock AgentCore. It provides a secure, fully managed, serverless hosting environment purpose-built for deploying, executing, and scaling AI agents and tools in production.

   It abstracts away all infrastructure management (scaling, cold starts, isolation, networking, etc.), so you can focus purely on your agent logic.

   - Main Purpose and Functionality
     - Hosts and runs agents/tools: Deploys your agent code (or MCP/A2A/AG-UI servers) as containerized applications that process user inputs, maintain context, reason, call tools, and generate responses.
     - Serverless execution model: Similar to AWS Lambda but optimized for agentic workloads. It uses lightweight microVMs for fast invocation.
     - Supports diverse workloads:
       - Real-time / low-latency conversations (fast cold starts).
       - Long-running and asynchronous agents (up to 8 hours).
       - Multi-modal inputs (text, images, audio, video) with up to 100MB+ payloads.
       - Multi-agent and agent-to-agent collaboration.
     - Framework and model agnostic: Works with any agent framework (LangGraph, CrewAI, LlamaIndex, Strands, custom code, etc.) and any foundation model (Amazon Bedrock, OpenAI, Anthropic, Google Gemini, Meta Llama, etc.).
     - Protocol support: Native support for HTTP, MCP (Model Context Protocol), A2A (Agent-to-Agent), and AG-UI (Agent-User Interaction) protocols for streaming, real-time UI updates, and tool interactions.
   - Core Components of AgentCore Runtime

Component	Description	Key Functionality

Component	Description	Key Functionality
AgentCore Runtime	Containerized execution environment for agents	Containerized execution environment with unique runtime per agent
Versions	Versioning system for agent deployments	Supports safe updates, rollbacks, blue/green deployments
Endpoints (Aliases)	Stable invocation endpoints for agents	Provides stable invocation URLs/ARNs; enables traffic shifting
Sessions	Maintains state across interactions	Maintains conversation state with true session handling
Execution Environment	Manages compute and isolation	Handles cold starts, resource allocation, security isolation
Built-in Tools Support	Integration with AWS tools and services	Can be configured for VPC access to private resources

- Key Features and Benefits
  - Security & Isolation:
    - Complete session isolation.
    - IAM execution roles for fine-grained permissions.
    - VPC connectivity for accessing private resources without public exposure.
  - Performance:
    - Fast cold starts for real-time responses.
    - Support for bidirectional streaming (SSE & WebSocket).
    - Handles large/multi-modal payloads efficiently.
  - Observability Integration:
    - Automatic metrics, traces, logs (invocations, latency, errors, token usage, CPU/memory).
    - Works seamlessly with AgentCore Observability and Amazon CloudWatch.
  - Deployment Options:
    - Direct code deploy (Starter Toolkit / CLI).
    - Container-based (Docker → ECR).
    - AWS Marketplace listings for reusable agents/tools.
  - Asynchronous & Long-Running Support:
    - Agents can start background tasks and respond immediately.
    - Users can check status later.
  - Protocol Contracts:
    - HTTP protocol for standard invocations.

        - MCP for tool discovery and calling.
        - A2A for agent-to-agent communication.
        - AG-UI for rich, real-time user interfaces.

    - How It Works (High-Level)

        1. You develop your agent locally using any framework.

        2. Deploy it to AgentCore Runtime (via CLI, console, or direct code upload) → AWS packages it into a container and deploys to a serverless environment.

        3. Create versions and endpoints for controlled rollout.

        4. Users invoke the endpoint (HTTP/MCP/etc.) → Runtime spins up an isolated execution environment → your agent processes the request, uses tools (via Gateway), memory, etc.

        5. Everything is automatically scaled, monitored, and secured.

    - Relationship with Other AgentCore Components

        - Gateway — Agents in Runtime call tools securely through the Gateway.
        - Memory — Provides short-term and long-term context to agents running in Runtime.
        - Identity — Handles authentication and secure credential passing.
        - Policy — Enforces rules on tool actions originating from Runtime.
        - Observability — Traces every step of execution inside Runtime.
        - Knowledge Bases — Commonly used as a retrieval tool by agents hosted in Runtime.

    AgentCore Runtime is where your agents actually "live" and run. All other AgentCore services (Gateway, Memory, etc.) are designed to complement agents hosted here.

2. **AgentCore Memory**
    Amazon Bedrock AgentCore Memory is a fully managed service that enables AI agents to maintain context-aware, personalized, and intelligent interactions by handling both short-term and long-term memory. It solves the stateless nature of most LLMs and agents by allowing them to remember past interactions, user preferences, facts, and experiences without you managing complex infrastructure (vector stores, summarization logic, consolidation, etc.).

    It works seamlessly with AgentCore Runtime and is compatible with popular frameworks like LangGraph, LangChain, Strands, and LlamaIndex.

    - Main Purpose and Functionality

        - Short-term memory: Stores raw conversation events/history within a session for coherent multi-turn interactions.

        - Long-term memory: Automatically extracts, structures, consolidates, and persists insights across sessions so agents can learn from users and improve over time.

        - Automatic processing pipeline: Converts raw events into searchable, structured memories using intelligent extraction and consolidation.

- Semantic retrieval: Agents can query relevant memories using natural language (vector-based similarity search).
- Multi-agent sharing: Multiple agents can read from or write to the same memory resource.
- Full control with zero ops overhead: You decide what to remember via configurable strategies, with built-in encryption, expiration, and observability.

This makes agents feel more human-like — they remember user details, avoid repeating questions, personalize responses, and evolve based on past experiences.

- Core Components of AgentCore Memory

Component	Description	Key Functionality
Memory Resource	Central memory system for agents	Central place that holds both short-term events and long-term knowledge
Short-term Memory	Session-level memory	Maintains immediate context within a session
Long-term Memory	Persistent memory across sessions	Stores insights that survive across sessions
Memory Strategies	Rules for extracting memory	Determines extraction logic; supports built-in and custom strategies
Built-in Strategies	Predefined extraction approaches	- Semantic: Extracts facts and concepts - User preference: Captures preferences - Conversation summary: Summarizes interactions
Extraction & Consolidation Pipeline	Processes and organizes memory	Extracts insights, merges related info, resolves conflicts
Namespaces / Organization	Logical grouping of memory	Organizes data by actor (user/agent), session, or application
Retrieval Interface	Access layer for memory	Write events → Retrieve relevant records (short-term & long-term)

- How It Works (High-Level Workflow)

  1. You create a Memory Resource and attach one or more strategies.
  2. During agent execution (in Runtime), your agent writes raw events (user messages, agent responses, tool results, etc.) to short-term memory.
  3. AgentCore Memory automatically triggers the extraction pipeline based on configured strategies → generates structured long-term memory records.

4. The agent later queries the memory (e.g., “Retrieve user preferences and past episodes related to this topic”) to augment its context before calling the LLM or making decisions.
5. Memories can be shared across agents and are fully observable via AgentCore Observability.

- Comparison with RAG (Knowledge Bases):
  - Short/Long-term Memory is personalized and experience-based (user-specific facts, preferences, history).
  - Knowledge Bases are for broad, external factual data (documents, enterprise knowledge).
- Key Features
  - Serverless & fully managed — No databases, embeddings, or cron jobs to handle.
  - Configurable retention & expiration for short-term events.
  - Custom strategies — Override or fully self-manage extraction logic if needed.
  - Security — Encryption (AWS KMS support), IAM access control, namespace isolation.
  - Observability — Metrics, logs, and traces for memory operations (write volume, extraction latency, retrieval hits, etc.).
  - Streaming support for memory records.
  - Episodic Memory (advanced) — Remembers not just what happened, but how the agent reasoned and the outcome.
- Relationship with Other AgentCore Components
  - Runtime — Agents hosted in Runtime easily read/write memory during execution.
  - Gateway — Tools can trigger memory updates or use memory context.
  - Identity — Memories can be scoped to authenticated users/actors.
  - Policy — Can govern what agents are allowed to store or retrieve from memory.
  - Observability — Tracks memory usage, extraction success, and query performance.
  - Knowledge Bases — Often used together (Memory for personalization + KB for external facts).

Note: AgentCore Memory is not a simple vector database. It includes intelligent extraction, consolidation, and multiple specialized memory types that go far beyond basic RAG.

3. **AgentCore Gateway**:
   Amazon Bedrock AgentCore Gateway is a fully managed service within the Amazon Bedrock AgentCore platform (launched/announced around mid-2025). It acts as a centralized, secure tool server that connects AI agents to real-world tools, APIs, data sources, and services. It primarily functions as a unified MCP (Model Context Protocol) server. MCP is an open standard (originally from Anthropic, now under the Linux Foundation) that standardizes how AI agents discover, select, and invoke tools.

   - Main Purpose and Functionality

- Transforms existing capabilities into agent-ready tools with minimal or zero code:
  - Converts REST APIs (via OpenAPI/Smithy specs or Amazon API Gateway stages).
  - Converts AWS Lambda functions.
  - Connects to existing MCP servers (daisy-chaining).
  - Provides 1-click integrations with popular enterprise tools (e.g., Salesforce, Slack, Jira, Asana, Zendesk, and others).
- Provides a single access point (MCP endpoint/URL) for agents to interact with multiple tools.
- Enables intelligent tool discovery:
  - List tools.
  - Semantic search for tools (via natural language queries).
  - Automatic tool description generation (name, description, input/output schemas).
- Handles security comprehensively (unique in being fully managed for both sides):
  - Inbound authorization (ingress): Controls which agents/users can access the gateway (e.g., via AgentCore Identity, OAuth 2.0, IAM).
  - Outbound authorization (egress): Securely passes credentials/tokens to downstream targets (e.g., API keys, OAuth flows, IAM for Lambda/Smithy). Uses a secure credential vault for per-actor, per-target credentials.
- Serverless and scalable: No infrastructure management; pay only for what you use. Abstracts protocol translation, authentication, and connectivity.
- Additional features:
  - Debugging support and detailed messages.
  - Interceptors (custom Lambda-based) for fine-grained access control, request/response modification, and dynamic schema management.
  - CloudTrail logging for auditing (management and data events like InvokeGateway).
  - Custom encryption options.

  Agents interact with the gateway using standard MCP operations (e.g., list_tools, invoke_tool, semantic tool selection). The gateway then translates these into calls to the underlying targets.
- Core Concepts
  - Gateway — The top-level resource. It acts as the managed MCP server with a unique endpoint. One gateway can aggregate many tools.
  - Gateway Targets — Individual backends that provide tools. A gateway can have multiple targets. Supported target types include:
    - OpenAPI schemas (REST APIs).
    - Smithy models.
    - AWS Lambda functions.
    - Existing MCP servers.
    - Amazon API Gateway stages.

        - Pre-built integration templates (e.g., for Salesforce, Jira, etc.).
    - AgentCore Identity — Often used together for centralized identity, credential management, and authorization (inbound + outbound).
    - Interceptors (optional) — Custom logic (via Lambda) that runs before/after tool invocation for advanced control.
  - How It Works (High-Level Workflow)
    1. You create a Gateway and add one or more Targets (pointing to your API schema, Lambda, etc.).
    2. The gateway automatically introspects the target and exposes it as MCP-compatible tools.
    3. Your AI agent (running in AgentCore Runtime or any MCP-compatible framework like LangGraph, CrewAI, etc.) connects to the gateway's MCP endpoint.
    4. The agent discovers tools (via list or semantic search), then invokes them.
    5. The gateway handles auth, translates the MCP call, invokes the real backend, and returns the result.

    This eliminates the need for custom glue code, manual hosting of MCP servers, or scattered security logic.
  - Benefits
    - Accelerates agent development by making any API/Lambda "agent-ready" quickly.
    - Centralizes tool management and governance.
    - Enterprise-grade security without operational overhead.
    - Works with any foundation model and many agent frameworks.

4. **AgentCore Identity**
   Amazon Bedrock AgentCore Identity is a centralized identity and credential management service designed specifically for AI agents and automated workloads. It handles both inbound authentication (who can invoke/call the agent) and outbound authentication (how the agent securely accesses external services and tools on behalf of users or autonomously).

   It solves key challenges in agentic AI: managing non-human (workload) identities, securely handling credentials/tokens without exposing them, supporting OAuth flows, and enabling agents to act with proper delegation while maintaining strict security and auditability.

   - Main Purpose and Functionality
     - Inbound Authentication (Ingress): Verifies and authorizes users, applications, or other agents that want to invoke an AgentCore Runtime agent or tool.
     - Outbound Authentication (Egress): Allows agents to securely obtain and use credentials (OAuth tokens, API keys, Sigv4, etc.) to call downstream services (AWS resources, third-party apps like Slack, Salesforce, Google Drive, GitHub, etc.) without hardcoding secrets.
     - Workload Identity Model: Agents get their own unique identities (not just user impersonation) with metadata, ARNs, and specialized attributes.

    - Secure Credential Handling: Stores and retrieves tokens/API keys in a vault with least-privilege access; supports user-delegated (with consent) and autonomous modes.
    - Seamless Integration: Works natively with AgentCore Runtime, Gateway, Policy, and external Identity Providers (IdPs) like Amazon Cognito, Okta, Microsoft Entra ID, Auth0, etc.
    - Protocol Support: OAuth 2.0 (Authorization Code, Client Credentials, etc.), Sigv4 for AWS, API keys, and MCP-compliant flows.

  This enables agents to act "on behalf of" users safely (with explicit consent where needed) while reducing token sprawl and security risks.

- Core Components of AgentCore Identity

Component	Description	Key Functionality
Agent Identity Directory	Stores and manages identities for agents	Create, manage, and organize unique agent identities
Agent Authorizer	Handles authentication and authorization logic	Performs authentication and authorization checks
Resource Credential Provider	Defines how credentials are obtained	Defines credential types (OAuth, API key, Sigv4, etc.)
Resource Token Vault	Secure storage for tokens and secrets	Holds user-delegated or workload tokens; allows secure retrieval

- Key Features
    - Dual-layer Security: Strong inbound controls + secure outbound credential delegation.
    - OAuth 2.0 Support: Full flows including Authorization Code (with user consent), Client Credentials (machine-to-machine), and token exchange.
    - Credential Scoping & Least Privilege: Fine-grained control over what credentials an agent can access and for which scopes.
    - Integration with Policy: Works together with AgentCore Policy for authorization decisions on tool calls.
    - Observability: Emits metrics, logs, and spans for identity operations (visible in AgentCore Observability and CloudWatch).
    - No Secret Management Overhead: Agents never store or manage long-lived secrets directly.
    - Supported Patterns:
        - User-delegated access (agent acts with user's consent).
        - Autonomous agent access (workload identity with its own credentials).
        - Hybrid scenarios.

- How It Works (High-Level Flow)

  Inbound:
  1. User/application calls an AgentCore Runtime endpoint.
  2. Agent Authorizer validates the request (JWT, Cognito token, etc.).
  3. If allowed, the request reaches the agent.

  Outbound:
  1. Agent needs to call an external tool/service (via Gateway or directly).
  2. Agent requests credentials from Resource Credential Provider.
  3. Resource Token Vault securely provides the appropriate token/API key (based on stored config and user consent).
  4. Agent uses the short-lived credential to call the target (e.g., Slack API, Google Drive).

  Everything is audited and integrates with AgentCore Observability.

- Relationship with Other AgentCore Components

  - Runtime — Uses Identity for inbound auth of hosted agents and outbound auth from running agents.
  - Gateway — Identity secures tool invocations (inbound to gateway + outbound to targets).
  - Policy — Combines with Identity to enforce "who can do what" on tools.
  - Observability — Tracks auth events, token usage, and authorization flows.
  - Memory — Can store session-bound identity context if needed.

Note: AgentCore Identity is not a replacement for AWS IAM. It complements IAM by focusing on agent-specific workload identities and external (often OAuth-based) credential management.

5. **AgentCore Observability** :
   Amazon Bedrock AgentCore Observability is a built-in observability capability within the Amazon Bedrock AgentCore platform. It provides comprehensive tracing, debugging, monitoring, and performance insights for AI agents and related resources (Runtime, Memory, Gateway, Identity, Policy, and built-in tools) in production environments. It helps you visualize every step of an agent's workflow, audit decisions and intermediate outputs, identify bottlenecks or failures, and monitor operational metrics such as latency, token usage, error rates, and session duration.

   - Main Purpose and Functionality

     - End-to-end tracing and debugging: Offers detailed visualizations of agent execution paths, reasoning steps, tool invocations, memory interactions, and decision-making processes.

     - Real-time monitoring: Provides dashboards and metrics for agent performance, resource utilization, and quality evaluation (e.g., correctness, helpfulness, safety, goal success rate).

     - Session-level insights: Tracks user interactions over time across sessions, including conversation flows and multi-agent workflows.

     - Telemetry emission: Automatically generates or supports OpenTelemetry (OTel)-compatible data (traces, spans, metrics, logs) for seamless integration with Amazon CloudWatch or third-party tools (e.g., Dynatrace, Instana, Arize, Weave).

    - Built-in vs. custom observability:
        - Default service-generated metrics and data for AgentCore-managed resources.
        - Custom instrumentation in agent code for deeper visibility (e.g., using AWS Distro for OpenTelemetry - ADOT).
    - Auditing and compliance: Helps audit agent decisions, troubleshoot issues quickly, and maintain quality at scale with rich metadata tagging and filtering.
    - Evaluation support: Continuous evaluation of agent quality across business criteria.

    AgentCore Observability integrates natively with Amazon CloudWatch (including the Generative AI Observability page and Transaction Search) and supports exporting telemetry to your existing observability stack.

- Core Concepts AgentCore Observability is structured around these key telemetry concepts and resource-specific data.
    1. Sessions (S):
        - Represents a complete user interaction or conversation with the agent.
        - Tracks overall duration, success/failure, and high-level metrics.
    2. Traces (T):
        - End-to-end records of a request or invocation flow.
        - Shows the full execution path across the agent, tools, memory, gateway, etc.
    3. Spans:
        - Individual units of work within a trace (e.g., LLM call, tool invocation, memory read/write, reasoning step).
        - Include timing, inputs/outputs, metadata, errors, and token usage.
    4. Metrics:
        - Aggregated performance indicators (batched at 1-minute intervals in most cases).
        - Examples: invocation count, latency (p50/p90/p99), error rates, token usage, session count, throttles, duration.
    5. Logs:
        - Detailed event logs for troubleshooting (e.g., execution logs, authorization events).
- Observability Data by AgentCore Component AgentCore automatically provides or supports observability for the following:
    - Runtime Observability:
        - Agent execution activity, processing latency, resource utilization, error types.
        - Session metrics for agents hosted in AgentCore Runtime.
    - Memory Observability:

        - Metrics on memory store/read/write operations.
        - Spans tracing memory generation/access relationships.
    - Gateway Observability (related to AgentCore Gateway):
        - Tool invocation metrics, latency, error rates, and authorization events.
        - Logs for gateway processes.
    - Built-in Tools Observability:
        - Invocation counts, latency, throttles, and execution details for tools exposed via the Gateway.
    - Identity Observability:
        - Usage, authorization, and resource access metrics.
        - Spans and logs for authentication/authorization flows.
    - Policy Observability:
        - Metrics and traces related to guardrails, safety checks, and policy enforcement.
    - Service-Generated Data:
        - Default metrics available in CloudWatch even without full custom instrumentation.
- How It Works (High-Level Setup)
    1. One-time CloudWatch setup: Enable Transaction Search in CloudWatch for full traces/spans.
    2. For AgentCore Runtime-hosted agents: Many metrics are automatic; add OpenTelemetry instrumentation for custom spans/metrics.
    3. For agents outside Runtime (e.g., Lambda, EC2, or other frameworks): Instrument your code with ADOT or compatible auto-instrumentors.
    4. View data:
        - CloudWatch Generative AI Observability dashboards.
        - CloudWatch Logs and Metrics.
        - Traces in Transaction Search.
        - Integrate with external platforms via OpenTelemetry.

You can enhance observability with custom headers for richer context in runtime, tools, or identity calls.

- Benefits
    - Accelerates debugging of complex agentic workflows.
    - Provides production-grade visibility without heavy custom coding.
    - Supports cost optimization (via token/latency metrics).
    - Enables proactive monitoring with alarms and anomaly detection.
    - Framework-agnostic and integrates with popular observability tools.

AgentCore Observability works hand-in-hand with other AgentCore services (Runtime for hosting, Gateway for tools, Memory for context, Identity for auth). It is essential for running reliable, production-grade AI agents at scale.

6. **AgentCore Policy** :
   Amazon Bedrock AgentCore Policy is a fully managed, centralized policy engine that provides deterministic, fine-grained control over what AI agents can and cannot do when interacting with tools. It acts as a protective layer that enforces rules outside the agent's code and before any tool is executed.

   It was made generally available in March 2026 and is designed to give security, compliance, and operations teams governance without requiring changes to agent logic or prompting.

   - Main Purpose and Functionality
     - Real-time enforcement of boundaries: Intercepts every agent-to-tool request at the AgentCore Gateway level and decides allow or deny instantly.
     - Deterministic authorization: Unlike LLM-based guardrails (which can be inconsistent), Policy uses strict, auditable rules that follow default-deny and forbid-wins semantics.
     - Fine-grained control: Decide which tools an agent can call, which actions are permitted, and under what conditions (based on user identity, input parameters, context, time, etc.).
     - Responsible AI & compliance: Prevent unsafe, harmful, or non-compliant actions (e.g., block large refunds, restrict access to sensitive data, enforce approval workflows).
     - No impact on agent code: Policies are defined centrally and applied at the gateway — agents remain unchanged.

     How it integrates: You attach a Policy Engine to an AgentCore Gateway. Every tool invocation (via MCP) goes through the policy engine for evaluation before reaching the actual tool (API, Lambda, etc.).

   - Core Components of AgentCore Policy

Component	Description	Key Role
Policy Engine	Evaluates every tool request in real-time	Evaluates every tool request in real-time. One central decision point
Policies	Define who can do what to which resource under what conditions	Define access control logic
Cedar Language	Provides mathematical analyzable, auditable, and safe policy definitions	Ensures secure and verifiable authorization
Natural Language Authoring	Makes policy creation accessible to non-developers	Simplifies policy creation

Component	Description	Key Role
Schema & Context	Enables context-aware decisions (e.g., “allow only during business hours”)	Supports fine-grained, context-aware authorization

- How Policy Evaluation Works (Authorization Flow)

  1. Agent decides to call a tool → request goes to AgentCore Gateway.
  2. Gateway forwards the request to the attached Policy Engine.
  3. Policy Engine evaluates all applicable policies:
     - Checks principal (agent/user identity),
     - Action (specific tool/method),
     - Resource (gateway or tool),
     - Conditions (when clause — input values, time, user attributes, etc.).
  4. Outcome:
     - Any forbid policy matches → DENY
     - At least one permit matches and no forbid → ALLOW
     - No permit matches → DENY (default-deny)
  5. If allowed, the tool executes; if denied, the agent receives a clear error.

  This happens with very low latency and is fully auditable via CloudTrail and Observability

- Relationship with Other AgentCore Components

  - Gateway — Policy attaches here and intercepts all tool traffic.
  - Identity — Provides principal and context (OAuth users, tags) for policy decisions.
  - Observability — Logs policy evaluations, allows, denies, and reasons.
  - Runtime — Agents running in Runtime benefit from Policy without code changes.
  - Memory — Can be used together for context-aware policies.

Note: AgentCore Policy is different from Amazon Bedrock Guardrails. Guardrails typically filter model input/output content (safety, toxicity, hallucinations). Policy focuses on tool/action authorization — what the agent is allowed to do in the external world.

- Important Notes

  - Knowledge Bases (Amazon Bedrock Knowledge Bases) is not a core AgentCore component. It is a separate Bedrock service commonly integrated with AgentCore agents for RAG (retrieval-augmented generation).
  - Evaluations is sometimes mentioned as an additional capability for testing and scoring agent quality.
  - All components are fully managed and serverless — you pay only for what you use.
  - They integrate natively with each other (e.g., Runtime + Gateway + Memory + Observability is a very common production stack).

- Typical Architecture Flow User → Runtime (Agent) → uses Memory for context → calls tools via Gateway → secured by Identity → everything monitored by Observability → governed by Policy.